IY3660 – Applications of Cryptography

Week 6 lecture
Applications of hash functions and MACs

Guest lecturer: Yiannis Tselekounis

Information Security Group

ROYAL HOLLOWAY UNIVERSITY OF LONDON

Overview of this lecture

- Applications of hash functions
  - Password hashing
  - PoW (proof of work) schemes
  - Others
- MAC algorithms
- Security of MAC constructions
- MAC constructions

(Some) applications of hash functions

(from Week 5)

Data fingerprinting

- Scenario 1
  - You want to protect the integrity of key system files (against hackers modifying them), by hashing the files and securely storing the hashes
  - The system regularly checks whether the hashes match
- Scenario 2
  - You want to distribute a program online, and users want to ensure its integrity before installation
  - You can then also distribute the hash via a (possibly low-bandwidth) authentic channel, and users can check that the hash matches

- **Security property?**
  - 2nd pre-image resistance

Password protection

- Scenario:
  - Authenticating systems, instead of storing *(username, password)*, store *(username, H(password))*
  - When attempting to log on, the system computes the hash of the password provided, and checks a match with stored (*username, hash*)
  - Note that any string *s* satisfying *H(s) = H(password)* will be considered as a valid password for the purpose of logging on to the system

- **Security property?**
  - Pre-image resistance

Commitment schemes

- You decided to run a competition, in which players need to guess the winner of the next World Cup
- Participants must guess and **commit** to that guess today
- Participants want to keep their choices private
- **First attempt**:
  - Ask them to submit their guesses as *H(winner)*
  - Then, once the World Cup ends, they can disclose their guess and prove it was correct!
- **Attack**:
  - Since there aren't that many countries, you could recover their guesses by running through all countries, computing the hash and comparing with their commitment

Commitment schemes

- You decided to run a competition, in which players need to guess the winner of the next World Cup
- Participants must guess and **commit** to that guess today
- Participants want to keep their choices private
- **Second attempt**:
  - Ask them to submit their guesses $H(winner \mid\mid r)$, where r is a k-bit random string
  - Then, once the World Cup ends, they can disclose their guess and the random string to prove that their country guess was correct!
  - Is this secure?

Password hashing

Password hashing

- Setting
  - Users want to remotely authenticate to server using username and password

- Naïve implementation: server has database with all passwords **in clear**
  - Security threats?

Username	Password
Alice	password
Bob	123456
…	…

Password hashing

Main threat: attacker breaks into the server, steals this database and then go on to impersonate users

- This is a very realistic threat
- Examples: RockYou, Adobe, Experian,…
- See https://informationisbeautiful.net/visualizations/worlds-biggest-data-breaches-hacks/

Solution: store only password **hashes** in database

Username	Password hashes
Alice	AB1134F3DF
Bob	A76642E3CD
…	…

Password processing and storage

- Modern systems do not usually store passwords in plaintext

- On enrolment, when selecting a password, the system will input the password into a one-way function (e.g., a hash function), and store the output
  - *H(password) = h*
  - *h* is stored

- When authenticating, user will enter a password, the system will use it as input to the same function, and compare with stored value
  - *H(password') = h'*
  - If *h' == h*, then authentication is successful; else failed!

- Note that we don't need *password' = password*; only that *h' = h*

Discussion: to what extent does this approach improve security?

- Security depends on one-wayness of hash function H used
  - If H can be reversed, then passwords can be found from hashes
- If password space is not too large, attacker does not need to invert H
  - Can use password crackers instead: build dictionary, try to hash all passwords in dictionary, check for matches
  - Use of GPUs and special-purpose hardware to speed-up password cracking
- Use of common passwords
  - Makes cracking many accounts easy
- Salting: store (salt, H(salt || password)) instead of H(password)
  - Salt is an account-specific random value
  - Effect on security?
- **Why hashing instead of encrypting?**

Password cracking: password recovery attacks

- We assume that online attacks are infeasible
  - Lock-out of accounts, slowdown of attempts, etc
- Then typical attacks are offline attacks
  - Given some password hash $h = H(p)$, recover p' such that $H(p') = h$
- **Brute force attack**
  - Try all strings (in valid/restricted alphabet) of length up to some n
  - If size of alphabet is A, then *time* complexity is on the order of A^n hash operations
  - Usually not feasible (real-time) if n and A are large

Some password statistics

Password type	%
numeric	19%
lower case alpha	42%
mixed case alpha	3%
mixed numeric alpha	30%
other charac	6%

Password length	%
≤ 7	37%
≤ 8	58%
≤ 9	70%

Source: Gildas Avoine. *Cracking Passwords With Time-memory Trade-offs*
https://sites.uclouvain.be/security/download/slides/Avoine-2013-irisa-slides.pdf

Password cracking: password recovery attacks

- **Dictionary attack**
  - Try all likely passwords up to some length *n*
  - Based on a dictionary file/database
  - Implementations may use heuristics to try more common/likely passwords first
- **Hybrid attack**
  - A combination of both approaches (brute-force and dictionary)
  - E.g., password123, password111, password2023, etc

- When attacking a system, one may not be interested in a single, particular password, but rather in any password in a large database
  - This increases the chances of success

Common passwords in *RockYou* breach (2009)

Rank	Password	Absolute Frequency	Relative Frequency
1	123456	0.008917	0.034250
2	12345	0.002425	0.059588
3	123456789	0.002355	0.01944
4	password	0.001824	0.009130
5	iloveyou	0.001532	0.007669
6	princess	0.001021	0.005111
7	1234567	0.000666	0.003455
8	rockyou	0.000641	0.003324
9	12345678	0.000630	0.003156
10	abc123	0.000511	0.001961

TABLE I: Frequencies of RockYou top 10 passwords.

- 32 million accounts, 14 million unique passwords (stored in cleartext!)
- Absolute frequency: fraction amongst all accounts
- Relative frequency: fraction amongst passwords of the same length
- Note presence of site-specific passwords, e.g “rockyou” ~ 15,000 > 0.06%

Real-time vs. precomputation attacks

- Two (extreme) approaches to try to reverse a one-way function
  - Hash function, or block cipher key recovery, etc
  - Real-time exhaustive search attack (brute force):
    - Time requirement: $T = A^n$
    - Memory requirement: $M = negligible$ (storage of, e.g., a single datapoint)
  - Table-based attack (precomputation):
    - Time requirement: $T = negligible$ (table lookup)
    - Memory requirement: $M=A^n$
    - Time required to compute table is usually **NOT** considered in the computation of the attack complexity (only its memory requirement)

Time-memory trade-off (TMTO) attacks

- A trade-off between the two extreme approaches:
  - Attacker performs precomputation
    - In a way which has reduced storage requirements
  - It then requires more time in the *real-time* attack phase
    - It still allows for a fast recovery
  - Based on the work of M Hellman (1980), for recovering a DES key
  - Idea behind rainbow tables for password cracking…
    - … and in fact of several advanced cryptanalytic attacks against cryptographic primitives

Design approaches: password salting

- Use a random account-specific value (the salt) in each hash calculation
  - Typically concatenate the salt with password, and input it into the hash
  - Some implementations make a salt-dependent modification in the structure of hash algorithm
- Salting means that each account has to be attacked individually
  - E.g., you would need a precomputed table for each different salt!
- Salt values are stored as a field in the database of usernames and password hashes
- 64 bits of salt is typical
  - Enough to prevent collisions in random choices of salt values (birthday paradox)

Username	Salt	Password hashes
Alice	B62477A8	AB1134F3DF
Bob	C2EF1377	A76642E3CD
…	…	…

H(`salt`||`password`)

Design approaches: hash iteration

- Hash functions are typically efficient – this can assist brute-force attacks
- Idea: **slow down** the password hash computation by iterating the hash function
- Choose an iteration method that makes parallelisation of attacks hard
- Obtain a trade-off between security and performance of authentication system
- Typical iteration count: 10,000

Username	Salt	Password hashes
Alice	B62477A8	AB1134F3DF
Bob	C2EF1377	A76642E3CD
…	…	…

$H^{10000}(\texttt{salt} \| \texttt{password})$

Design approaches: dedicated hash functions

- Specifically design hash functions that are **slow** and **hardware unfriendly**
- Many proposals, including:
  - scrypt (RFC 7914)
  - Argon2 (winner of the Password Hashing Competition, see https://password-hashing.net)
- Typically require large amounts of memory when computing the hash
  - Known as memory hard hash functions
- Also applicable in proof-of-work schemes for cryptocurrencies

Username	Salt	Password hashes
Alice	B62477A8	AB1134F3DF
Bob	C2EF1377	A76642E3CD
…	…	…

scrypt(`salt` || `password`)

Design approaches: encryption

- Why not just encrypt the passwords?
- Server decrypts entry from database and compares to password sent by user
- Needs careful selection of encryption method and management of key
  - Key needs to be always available and heavily protected

Username	IV	Encrypted passwords
Alice	B62477A8	AB1134F3DF
Bob	C2EF1377	A76642E3CD
…	…	…

$\mathrm{Enc}_{K,IV}(\texttt{password})$

Real-world example: UNIX (1970s)

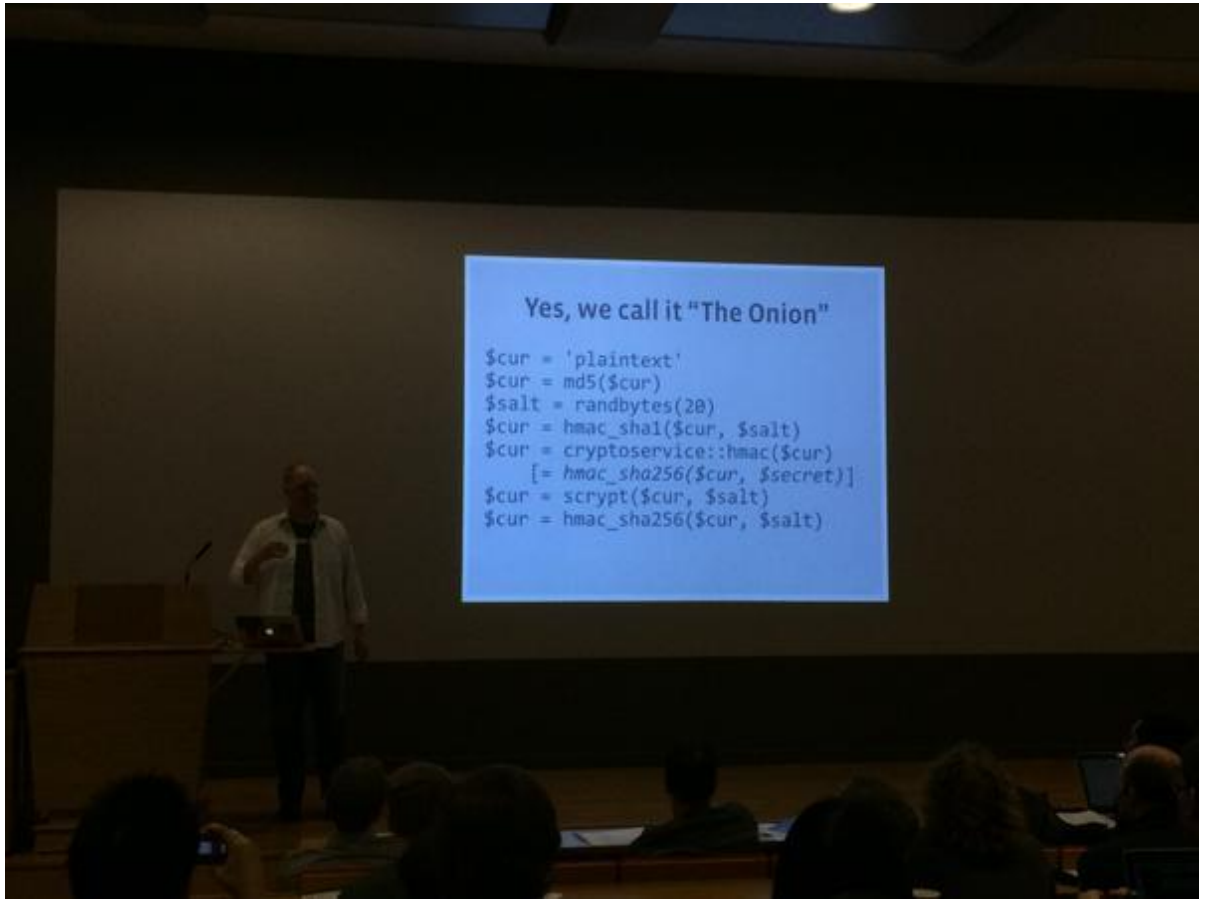
- Traditional DES-based password hashing in UNIX (from the 1970s)
- Password up to 8 characters (extra characters are ignored)
- Converted into a 56-bit DES key
- A 64-bit block of all zeros is encrypted with DES with the key derived from password
- The 64-bit output is converted into string of 11 printable characters
- DES is iterated 25 times
- 12-bit salt is used to modify the DES S-boxes (the structure of the algorithm)
- Somewhat modern in its approach:
  - DES is not too efficient in software
  - Salt made economically unviable to construct dedicated hardware
  - See “Password Security: A Case History” (Morris and Thompson, 1974)
- Modern Unix systems (Linux) use more modern algorithms/designs: e.g., SHA-2, bcrypt

Real-world example: Oracle database

- Password is converted to upper-case
- Username is concatenated with password
- Username could be considered as salt
  - Does not provide the same protection
  - E.g., the SYSTEM user in all systems use the same salt
- Resulting string goes through a (modified) DES-CBC computation

See: An Assessment of the Oracle Password Hashing Algorithm (Wright and Cid, 2005)

Real-world example: Facebook password onion



- Alec Muffet from Facebook explained their approach to password protection at RealWorldCrypto 2015
- Facebook have multiple layers of protection, mostly for legacy reasons
- MD5 + HMAC_SHA1 + Encryption + scrypt + HMAC_SHA256 (!!!)

Further reading

In addition to the suggestions in Moodle, see also:

- https://en.wikipedia.org/wiki/PBKDF2
- https://en.wikipedia.org/wiki/Time/memory/data_tradeoff_attack
- https://en.wikipedia.org/wiki/Rainbow_table
- https://project-rainbowcrack.com/table.htm

- PHC: https://password-hashing.net/

"The Password Hashing Competition (PHC) is an effort organized to identify new password hashing schemes in order to improve on the state-of-the-art (PBKDF2, scrypt, etc.), and to encourage the use of strong password protection. Applications include for example authentication to web services, PIN authentication on mobile devices, key derivation for full disk encryption, or private keys encryption."

Proof-of-Work

PoW: Proof-of-Work

- Proposed by Dwork and Naor (1992) to protect against spam or DoS
- Idea: the sender has to prove that it has spent some non-trivial amount of (computational) effort to send a message
- Example of a simplified version of the application:
  - Alice wishes to send a message m to Bob
  - Bob sends her a fresh challenge c
  - Alice must perform some computation based on c before sending m
  - This computation consumes some (non-trivial) computational resources, and there should be no obvious speed-up shortcuts for the computation
  - Alice sends the message with the result, which Bob (efficiently) verifies before accepting it

PoW: spam protection application

- Computation is not too expensive for a single message
  - It can be computed and verified reasonably efficiently
- But sending millions of messages consumes a lot of resources
  - Not only time, but also energy
- The challenge space must be large enough so that Alice cannot precompute/reuse the response

PoW: Hashcash

- The most popular PoW system proposed by Adam Back (1997)
- Hashcash is a cryptographic hash-based proof-of-work algorithm, where the task is to find **partial preimages**
- In the spam protection system, the sender attaches to the email a header, containing (among other information) the email recipient address, a timestamp and a nonce
  - This header is the input of a hash function
- The sender keeps varying the nonce until it can produce a hash output such that the first k bits are 0
  - That is, it looks for (particularly structured) partial preimage of 000…000****** (k zeros followed by * which can be any value {0,1})
  - k represents the *difficulty* of the task
- Receiver accepts message if the hash of header (with the particular nonce used by the sender) satisfies this property
- Verification very efficient: only one hash computation

PoW: in Bitcoin

- The PoW in bitcoin (a version of hashcash) uses SHA-256
- When **mining** bitcoins, miners need to find an input such that the hash output is lower than a particular value (the **target**)
  - Input has some structure and values from the blockchain and new transactions, but also a nonce
- You can roughly map the **target** to *k*
  - Essentially, the number of zeros at the start of the hash output
- These values are selected so that a new block is added to the blockchain on average every 10 minutes
- The target/difficulty keeps being adjusted based on the current hash power of the mining pool: https://learnmeabitcoin.com/technical/target
- The incentive for miners: if they win the race to mine the next block, they are rewarded with newly minted bitcoins

PoW in Bitcoin: some issues

- Very large energy consumption (wasted) in mining
  - An enormous number of hash computations (by millions of miners), but only one winner every 10 minute
  - Other cryptocurrencies are considering other schemes for achieving distributed consensus, e.g., Proof-of-Stake PoS
- Mining moved from CPUs, to GPUs, to FPGAs to ASICs
  - Now the only way to have a chance of mining a block is to have (expensive, specially designed) mining hardware
- Other cryptocurrencies use memory-hard hash functions functions to restore a fair playing field

Hash functions: further applications

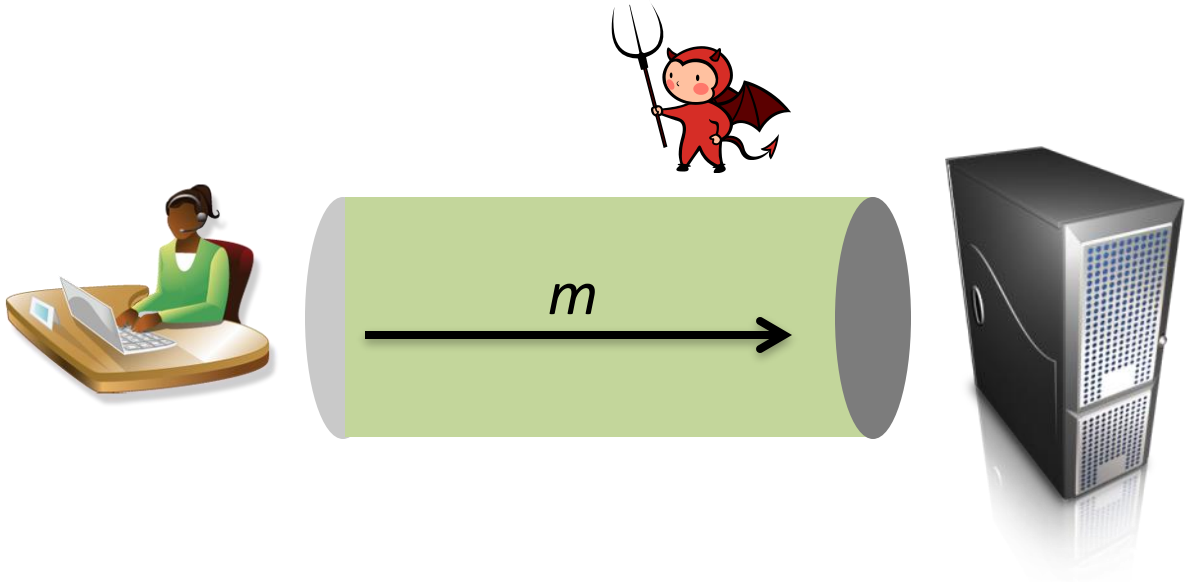
Further applications of hash functions

- MAC algorithms (we will see these next) can be based on hash functions
- Digital signature schemes (we will see these in a future lecture) use hash functions and rely heavily on **collision-resistance** for security
- Key derivation
  - Derive one key from another key
  - Commonly use hash functions to do so, using a pseudo-random function is more appropriate
- Commitments
  - Many cryptographic protocols need participants to commit to values
  - Can do so by sending hash of chosen value to other participants
  - Relies on **one-wayness** and **collision-resistance**

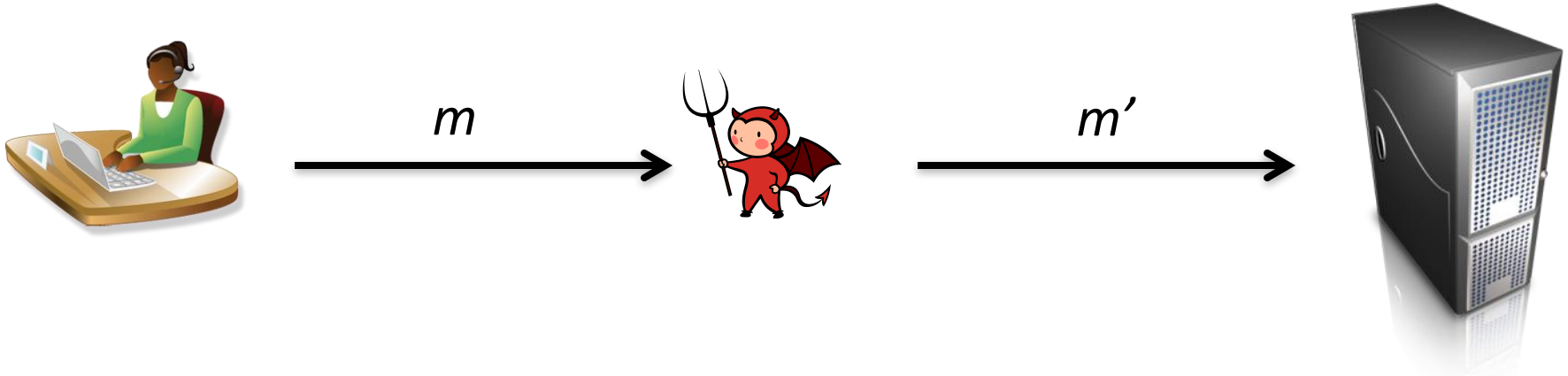
Message Authentication Codes

Introducing MACs

- Setting: assume Alice wants to send a message *m* to Bob
- Adversary controls the network link and wishes that Bob accepts a modified message *m'*
- Alice and Bob need a cryptographic primitive that can detect message modifications
- Primitive should provide integrity and data origin authentication of messages:
  - Assurance that messages have **not** been **tampered** with in transit
  - Assurance that messages **originate** from the expected sender
- In other words, no adversary should be able to forge messages



Introducing MACs



- Note that an attacker does not need to have nation-state resources to redirect and manipulate traffic, nor does it need to run the network (ISP)

- For example, an attacker can impersonate a public WiFi access point, or spoof DNS domain name resolution to redirect traffic and manipulate it

MACs vs encryption

c

$c = E_K(m)$

$m = D_K(c)$

- If Alice encrypts m, does she still need to protect its integrity?
- i.e., is it true that if message decrypts “correctly” then it has not been changed?

MACs vs encryption

$c = E_K(m)$

c

c'

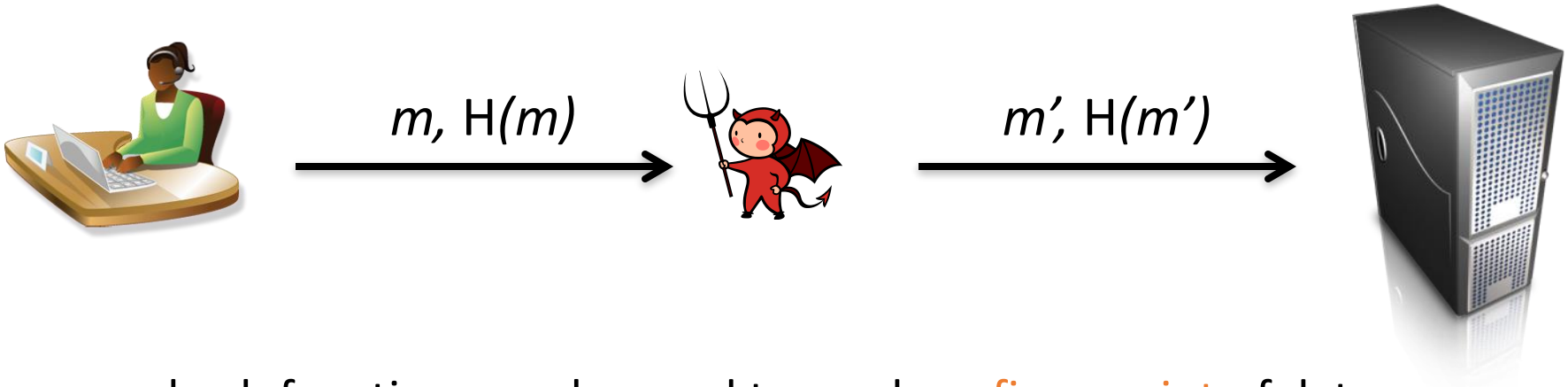
$m' = D_K(c')$,

with m' possibly "meaningful"

- Fallacy of "encryption provides message integrity"
- Recall attacks on stream ciphers and modes of operation
  - Bit changes in stream cipher ciphertext can't be detected
  - ECB-encrypted messages can have blocks re-ordered, deleted, etc
  - Attacks against CBC (changes in IV, etc)

Ordinary encryption **does not** provide message integrity!

MACs vs hash functions



- As we saw, hash functions can be used to produce fingerprint of data
  - Can't Alice simply send the hash of the message, H(m), along with m?

- Hash functions are public and unkeyed
  - The adversary can change the message and produce a new hash H(m') before forwarding it to Bob

- Integrity vs data origin authentication…

Authentication tags

- We need dedicated algorithms for providing **data integrity** and **data origin authentication**

$\tau \leftarrow \mathrm{Tag}_K(m)$

m, τ →

m', τ' →

Accept m' as being valid if
$\mathrm{Vrf}_K(m', \tau') = \texttt{true}$,
and reject otherwise

MAC – Message Authentication Codes

- We assume that Alice and Bob share a common secret key K
  - a **MAC is a symmetric cryptographic primitive**
- Using a dedicated algorithm *Tag*, Alice computes an **authentication tag** $\tau \leftarrow Tag_K(m)$ for each message m that she wishes to send to Bob
  - We often call τ the MAC or MAC value instead of authentication tag
- Alice now sends (m,τ) to Bob, instead of just m
- Bob verifies the integrity and data origin authenticity of the received message/tag pair (m',τ') by using a corresponding *Vrf* algorithm
  - *Vrf* algorithm returns either `true` or `false`
- Bob accepts only pairs for which:

$$\mathrm{Vrf}_K(m',\tau') = \texttt{true}$$

- **Security**: without knowing K, it should be hard for any adversary to create a pair (m',τ') such that $\mathrm{Vrf}_K(m',\tau') = \texttt{true}$
  - I.e., it should be hard to forge messages

Defining MACs

Definition:

A MAC algorithm with key length k and tag length t consists of two efficient, keyed algorithms:

$$Tag: \{0,1\}^k \times \{0,1\}^* \dashrightarrow \{0,1\}^t,$$

$$Vrf: \{0,1\}^k \times \{0,1\}^* \times \{0,1\}^t \dashrightarrow \{0,1\}.$$

For correctness, we require that for all K and all m, if $\tau \leftarrow Tag(K,m)$, then $Vrf(K,m,\tau) = 1$

That is, *Tag* takes a k-bit key and a message of arbitrary length, and produces a t-bit authentication tag;

Vrf takes a k-bit key ,a message of arbitrary length and a t-bit tag, and outputs 1 or 0 (`true` or `false`)

Notes:

- We assume keys K are generated as uniformly random k-bit strings
- We often write $Tag_K(m)$ for $Tag(K,m)$ and $Vrf_K(m,\tau)$ for $Vrf(K,m,\tau)$
- Both *Tag* and *Vrf* can be randomised, but typically are deterministic
- Correctness is a basic functionality requirement and **not** a security requirement!

Security of MACs

Security for MACs

Definition:

A MAC algorithm with key length k and tag length t consists of two efficient, keyed algorithms:

$$Tag: \{0,1\}^k \times \{0,1\}^* \dashrightarrow \{0,1\}^t,$$

$$Vrf: \{0,1\}^k \times \{0,1\}^* \times \{0,1\}^t \dashrightarrow \{0,1\}.$$

For correctness, we require that for all K and all m, if $\tau \leftarrow Tag(K,m)$, then:

$$Vrf(K,m,\tau) = 1$$

MAC unforgeability (informal):

It should be "hard" for an adversary who does not know the key K to forge a valid tag on a message, that is, to come up with a pair (m',τ') for which $Vrf(K,m',\tau') = 1$

- Even when the adversary has seen many message/tag pairs (m,τ) computed using the *Tag* algorithm with the same key K
- Even when the adversary gets to choose all the messages

Security for MACs

- The security definition means that the MAC tag τ must depend on every bit of the message m

Insecure example: $\mathrm{Tag}_K(m) = E_K(m_0 m_1 \ldots m_{t-1})$

  - tag only depends on first t bits of message

- The security definition also means that it must be hard to recover K given some pairs (m,τ), otherwise attacker can recover K, and then forgery is trivial

Insecure example:

$\mathrm{Tag}_K(m) = H(m) \oplus K$ where H is a hash function with output size k bits

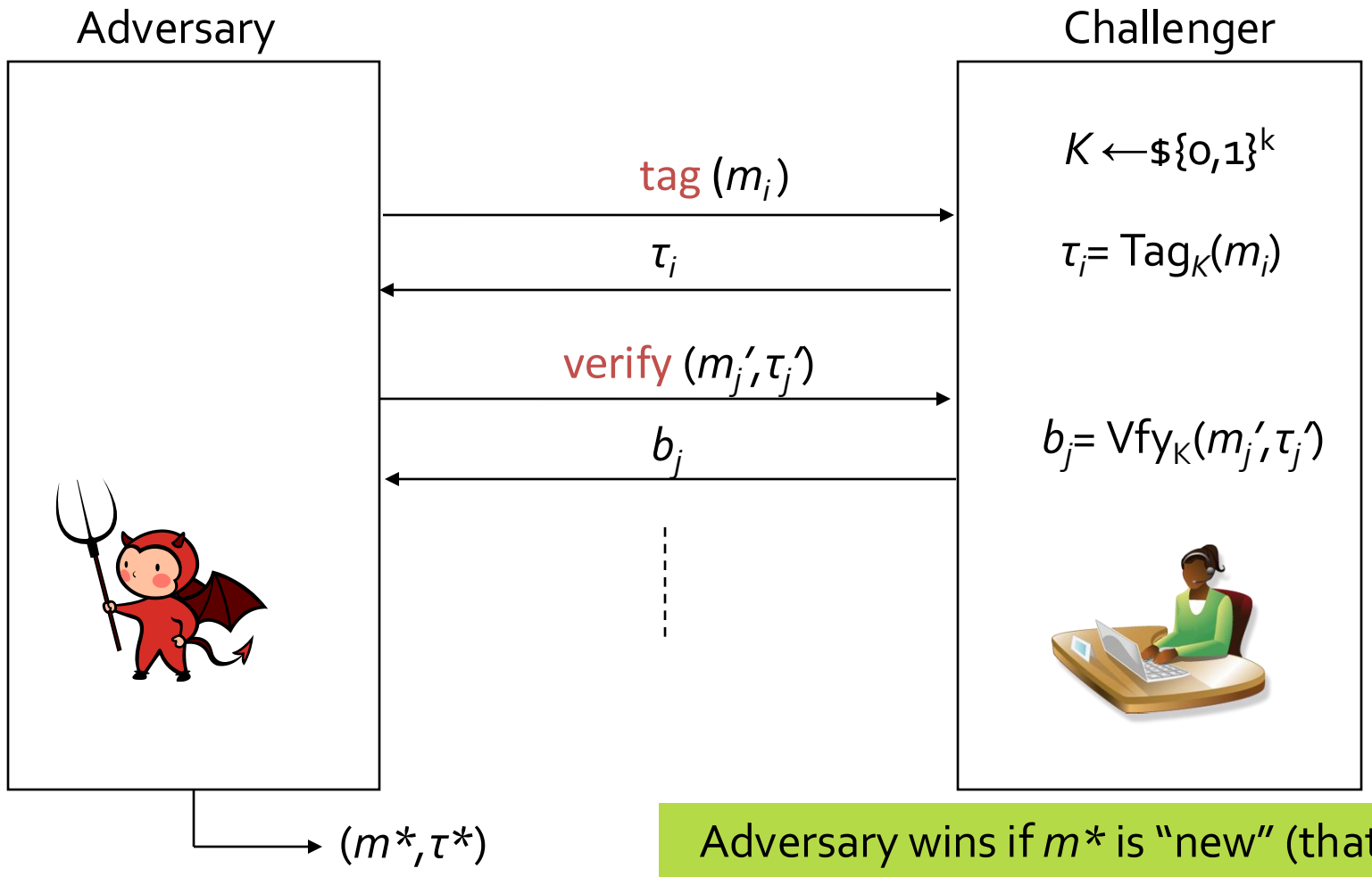
- NB: this is a necessary, but not sufficient condition:
- There can be attacks on MACs that forge MAC tags without recovering the key

Formalising security for MACs

Security of a MAC scheme (Tag,Vrf) is formalised as a game between an adversary and a challenger.

- The challenger begins by choosing a uniformly random key K
- The adversary has access to a tag oracle and a verify oracle
  - Tag oracle accepts messages as input and returns corresponding tags: $\tau \leftarrow \mathrm{Tag}_K(m)$
  - Using the tag oracle, the adversary can obtain MAC tags for messages of its choice: **a chosen message attack**
  - The verify oracle accepts pairs (m,τ) as input and returns $\mathrm{Vrf}_K(m,\tau)$
  - Adversary can use verify oracle to test whether a given (m,τ) is valid or not
- After accessing the tag and verify oracles as much as it likes (and doing any computation it desires), the adversary outputs a final pair (m^*,τ^*)

MAC security game (UF-CMA)



tag (m): interaction with tag oracle

verify (m', τ): interaction with verify oracle

Adversary wins if m^* is "new" (that is, not among the messages sent to the tag oracle) and $\mathrm{Vfy}(m^*, \tau^*) = 1$.

UF-CMA security

We say that a MAC scheme (Tag,Vrf) is **UF-CMA secure** (*Unforgeability under Chosen Message Attack*) if there is **no efficient** adversary that wins this game with a **significant probability.**

- "**Efficient**" here refers to the amount of time and memory used by the adversary and the number of queries it makes to its oracles

- **Significant probability?**
- One can always guess the t-bit tag value for any message, so there's an adversary A_G that wins with probability 2^{-t}, making no queries at all: A_G just outputs (m^*,τ^*) for any m^* and a random choice of τ^*
- An adversary that makes q queries of the form verify(m, τ) for random values of τ will receive a "1" response with probability at least $q.2^{-t}$; this adversary can then win either game with probability $q.2^{-t}$
- "Significant probability" can be formalised, but we won't do so here
- **Informally**: it means "not much better than these guessing adversaries do, averaged over random key K and adversary's randomness"

Typical MAC parameters

Key length *k*:

- Typically k larger than 80 to prevent **offline** exhaustive key search attacks (recall block cipher attacks)
- k = 128 for a comfortable security margin

Tag length *t*:

- *t* needs to be large enough to make guessing probability $q.2^{-t}$ sufficiently small, where *q* is number of verify queries
- Often quite small values are acceptable in practice, e.g., *t = 64* or 128
- Intuition: for adversary to make a verify query in practice, it needs to interact with Bob
- That is, attacks involving verify queries are inherently **online**
- But then Bob can limit the number of invalid MAC tags he accepts
- Hence *q*, the number of verify queries, can be kept small, so *t* need not be so large
- TLS works this way: any invalid MAC tag leads to connection drop and rekeying

Attacks that MACs do not prevent

Caution!!!

Secure MACs do provide integrity and data origin authentication, but…

- MACs cannot detect message deletion, message replay, or message reordering attacks
  - Additional mechanisms (e.g., sequence numbers) are needed in conjunction with MACs to provide these services
- If the same MAC key K is used in both communication directions, then MACs cannot prevent reflection attacks
  - Here, a pair (m,τ) created by A and sent to B is sent back to A by the adversary
  - A accepts (m,τ) as having come from B, when in fact it was generated by A!
  - Can prevent such attacks using directional indicators encoded in messages or using key separation (different keys for different purposes)
- MACs do not provide any kind of confidentiality guarantees
  - Need to use them in careful combination with encryption schemes to get both integrity and confidentiality

Two flavours of unforgeability

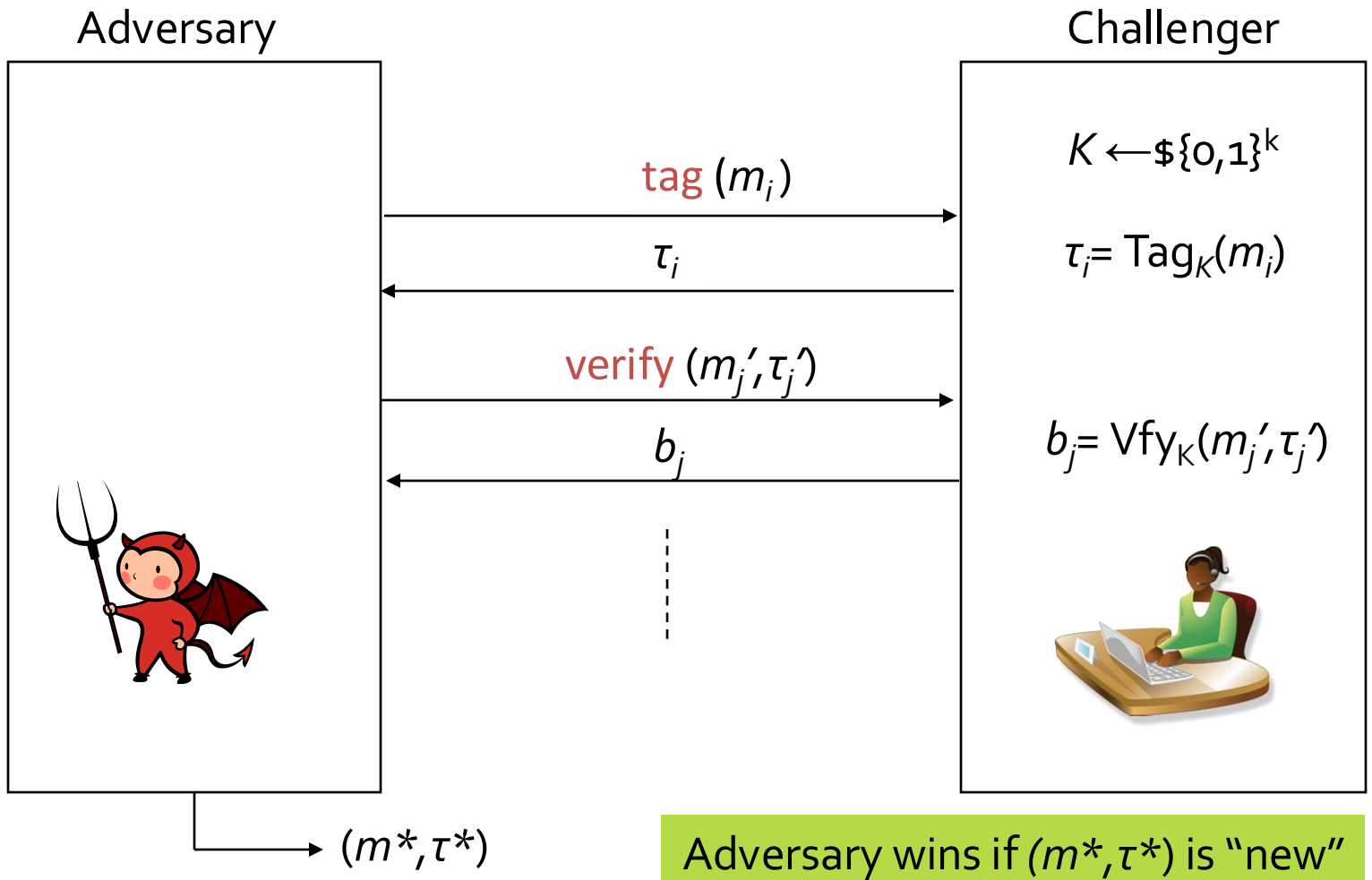
Sometimes called weak unforgeability and strong unforgeability

- In weak unforgeability, the adversary wins if m^* is distinct from all the m values involved in its tag oracle queries and if $\mathrm{Vrf}_K(m^*,\tau^*) = 1$
- In strong unforgeability, the adversary wins if (m^*,τ^*) is distinct from all the pairs (m,τ) used in its tag oracle queries and if $\mathrm{Vrf}_K(m^*,\tau^*) = 1$

A successful strong adversary might be able to win by coming up with a different valid tag τ^* on a message m for which it queried its tag oracle and received a tag τ

- That is, the adversary builds (m,τ) from one of its tag queries and its final output is (m,τ^*) where $\tau \neq \tau^*$
- Such an adversary would not win in the weak game (because it queried m to its tag oracle)

MAC: strong unforgeability game



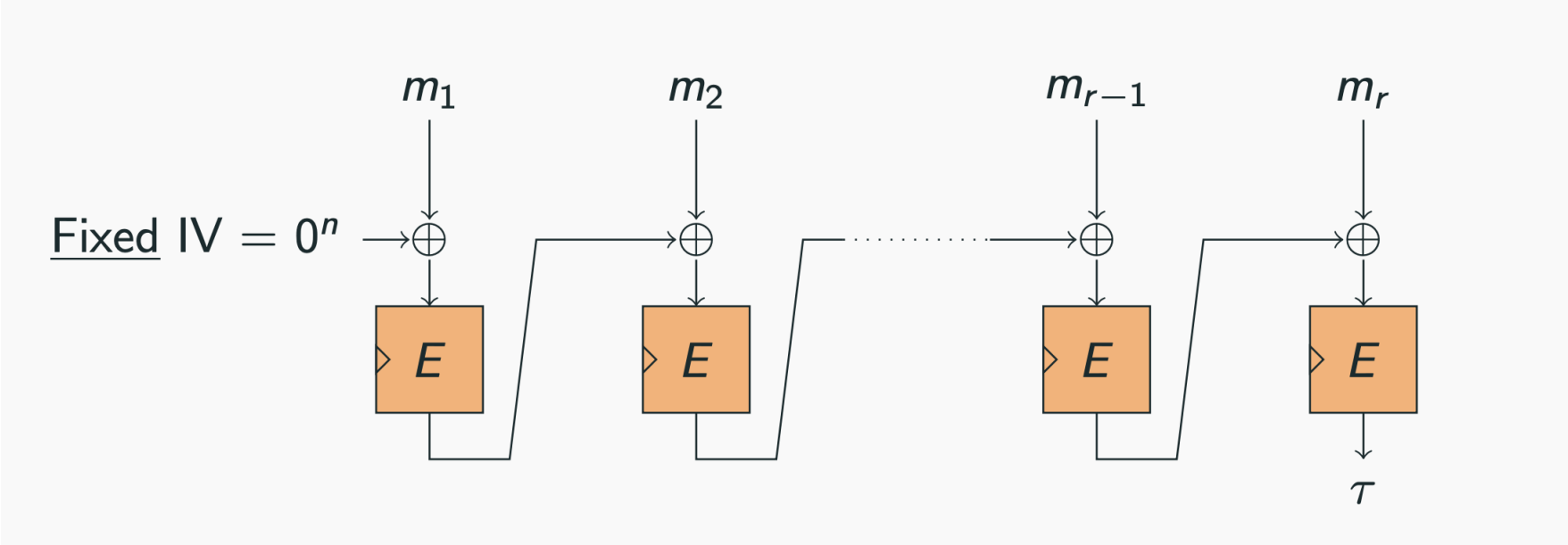
tag (m): interaction with tag oracle

verify (m', τ): interaction with verify oracle

Adversary wins if (m^*,τ^*) is "new" and $\mathrm{Vfy}(m^*, \tau^*) = 1$.

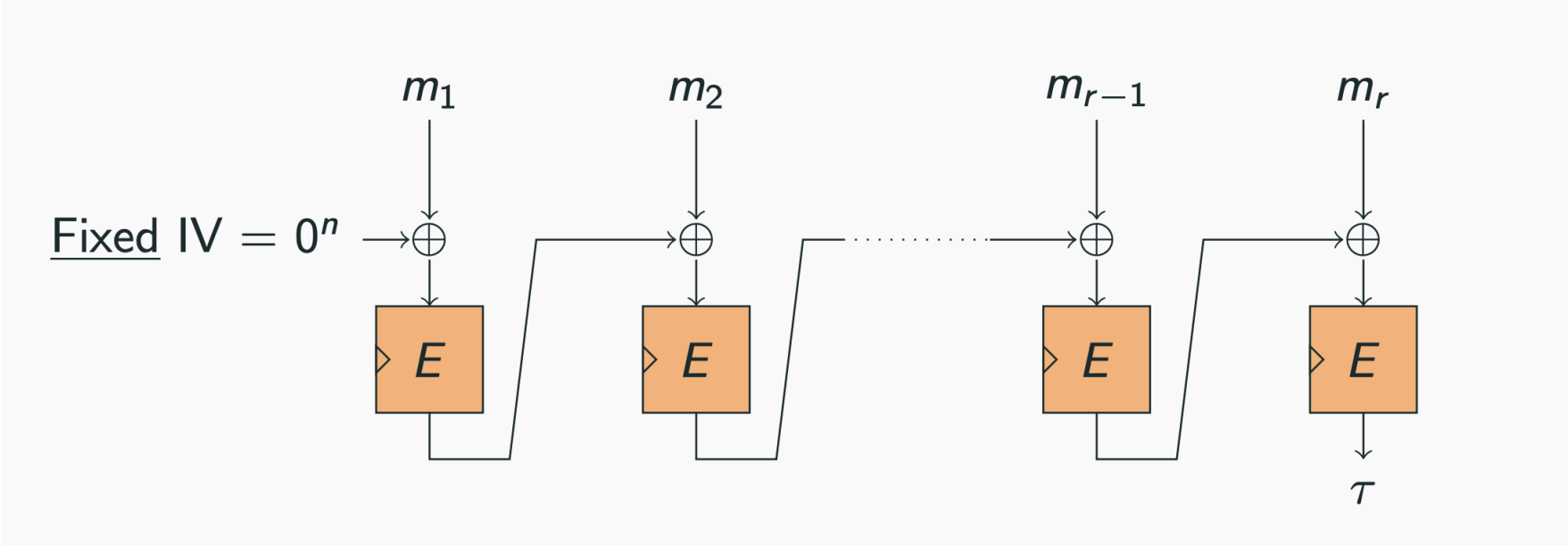
A MAC construction

Based on block ciphers: CBC-MAC



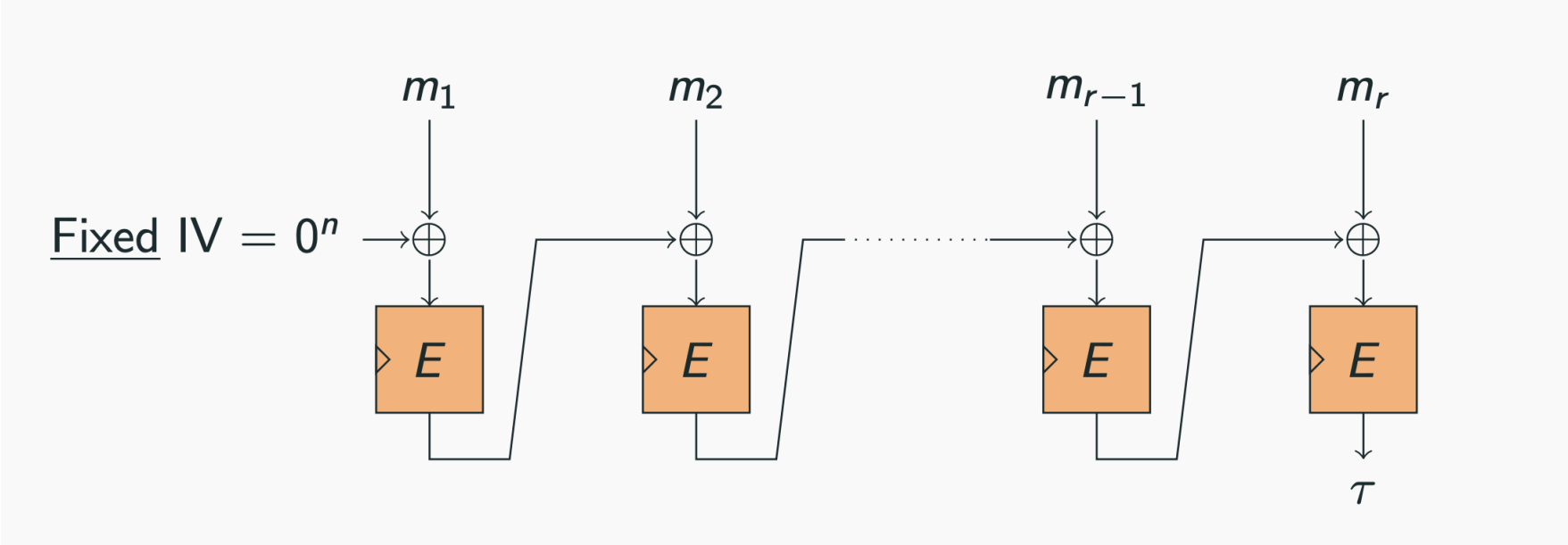
- Let E be an n-bit block cipher with fixed key K
- Split message m into n-bit chunks $m_1, \ldots, m_r$
  - Pad message if necessary
- Tag is computed as shown

Based on block ciphers: CBC-MAC



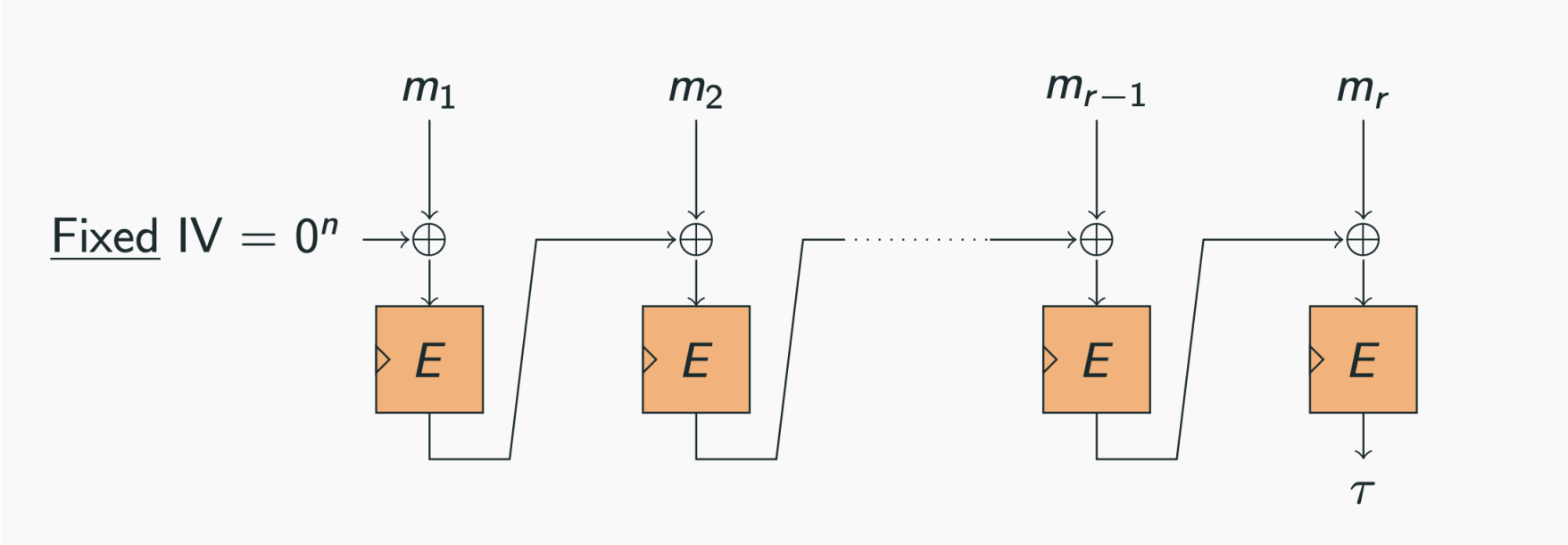
- Similar to CBC encryption, but
  - IV is fixed
  - Only output the final "ciphertext" block as the tag
- Tag has n bits but can be truncated if necessary

Security of CBC-MAC



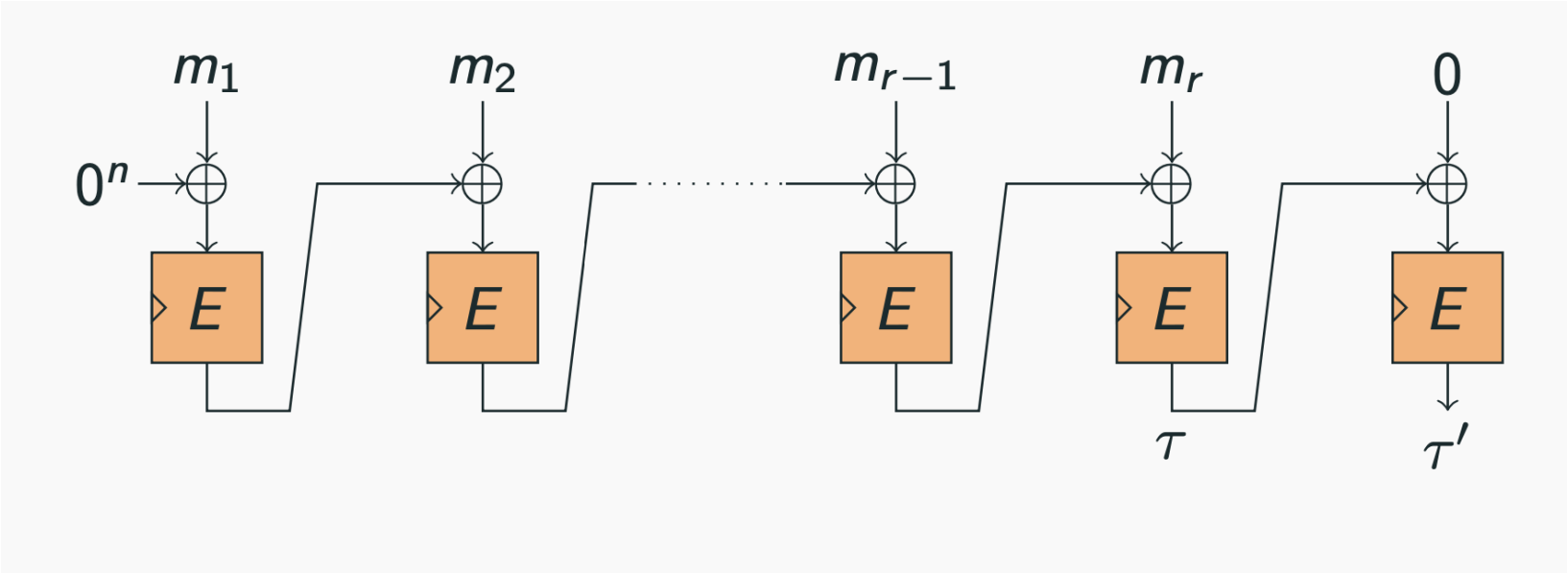
- How hard is it to produce forgeries in the UF-CMA model?

Security of CBC-MAC



- Recall that tag $\tau = \mathrm{Tag}(m)$ has n bits
- Set new message $m' = \tau$
- What is the MAC tag $\tau' = \mathrm{Tag}(m')$ of m'?
- $\mathrm{Tag}(m') = E_K(0^n \oplus m') = E_K(0^n \oplus \tau) = E_K(\tau) = \tau'$

Security of CBC-MAC



- Given a MAC tag for m and τ we can produce a forgery without knowledge of K
- Consider the message $m^* = m||0^n$ and tag τ^*= τ'
- The pair (m^*, τ^*) is a valid forgery!
  - $\mathrm{Tag}(m^*) = \mathrm{Tag}(m||0^n) = E_K(0^n \oplus \tau) = \mathrm{Tag}(\tau) = \tau'$

Security of CBC-MAC

- We saw a forgery attack against CBC-MAC
  - This shows CBC-MAC is not UF-CMA secure!

- Original message had $r \cdot n$ bits and forged message had $(r + 1)n$ bits

- The simplest way to secure CBC-MAC (as is) is to prepend the message length to the message before computing the MAC

Homework

- Read:
  - Chapters 6 and 8.4 from *Everyday Cryptography*
  - Chapter 7 of *Serious Cryptography*
- Go through the further reading suggestions